v1.0.0
20260216

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 1.2]

MODELLING CONCURRENT PROGRAM EXECUTION

SUMMARY

- Making models of concurrent programs
  - State diagrams & execution scenarios
- Correctness
  - safety and liveness properties
  - fairness
- Critical Section Problem
  - path towards Dekker's algorithm
- Verification of Concurrent Programs
  - linear temporal logic - LTL

FROM PROGRAMS TO MODELS (AND BACK)

- Importance of models and abstraction for computer science and engineering in particular
  - model: rigorous description / representation of program (system) structure and behaviour at a proper level of abstraction
    - including relevant information, abstracting from non-relevant aspects
  - diagrammatical representations for program design
  - formal models for program analysis and verification
- Defining proper models for concurrent programs
  - defining models for the structure and behaviour of concurrent programs abstracting from the low-level details of their actual implementation and realisation
    - design
  - enabling the possibility to reason about their dynamic behaviour of concurrent programs
    - verification

A MODEL FOR CONCURRENT PROGRAM EXECUTION

- Modelling each process as a sequence of **atomic actions**, each action corresponding to the atomic execution of a statement
- ***Speed independence assumption* =>**

  modelling the execution of a concurrent program as a sequence of actions obtained by **arbitrarily interleaving** the actions (atomic statements) from the processes
  - a single abstract *global processor* executing all the actions
  - **atomic** statements => executed to completion without the possibility of interleaving
  - during the computation the *control pointer* or instruction of a process indicates the next statement that can be executed by that process
- a **computation** or **scenario** is an execution sequence that can occur as a result of the interleaving

FIRST TRIVIAL EXAMPLE

`integer n := 0`	
p	**q**
`integer k1 := 1` **`p1:`** `n := k1`	`integer k2 := 2` **`q1:`** `n := k2`

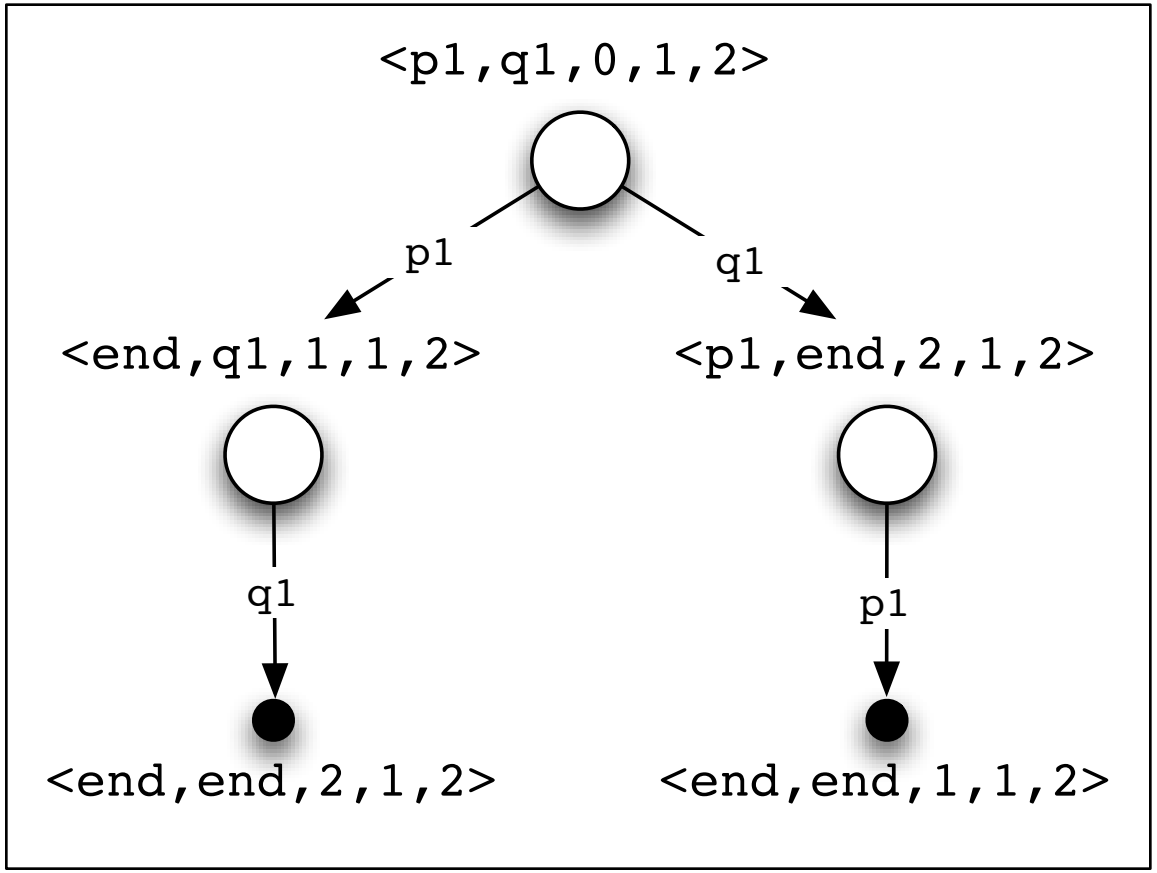
- Each labeled line represents an atomic statement
- Each process has private memory
  - local variables, such as k1 and k2
- Processes shares some memory
  - global variables, such as n
- Program execution: 2 scenarios
  - p1, q1 (=> n finally is equal to 2)
  - q1, p1 (=> n finally is equal to 1)

STATE DIAGRAMS

- Given the model, the execution of a concurrent program can be formally represented by **states** and **transitions** between states
  - the state is defined by a tuple consisting of
    - one element of each process that is a label (statement) from that process
    - one element for each global or local variable that is a value whose type is the same as the type of a variable
  - there is a transition between two states s1 and s2 if executing a statement in state s1 changes the state to s2
    - the statement executed must be one of those pointed to by a control pointer in s1
- The **state diagram** is a *graph* containing all the *reachable states* of the programs
  - scenarios are represented by directed paths through the state diagram from the initial state
  - cycles represent the possibility of infinite computation in a finite graph

FIRST EXAMPLE: STATE DIAGRAM

- State tuple: `<p,q,n,k1,k2>`



- 5 states, 2 scenarios

EXAMPLE #2

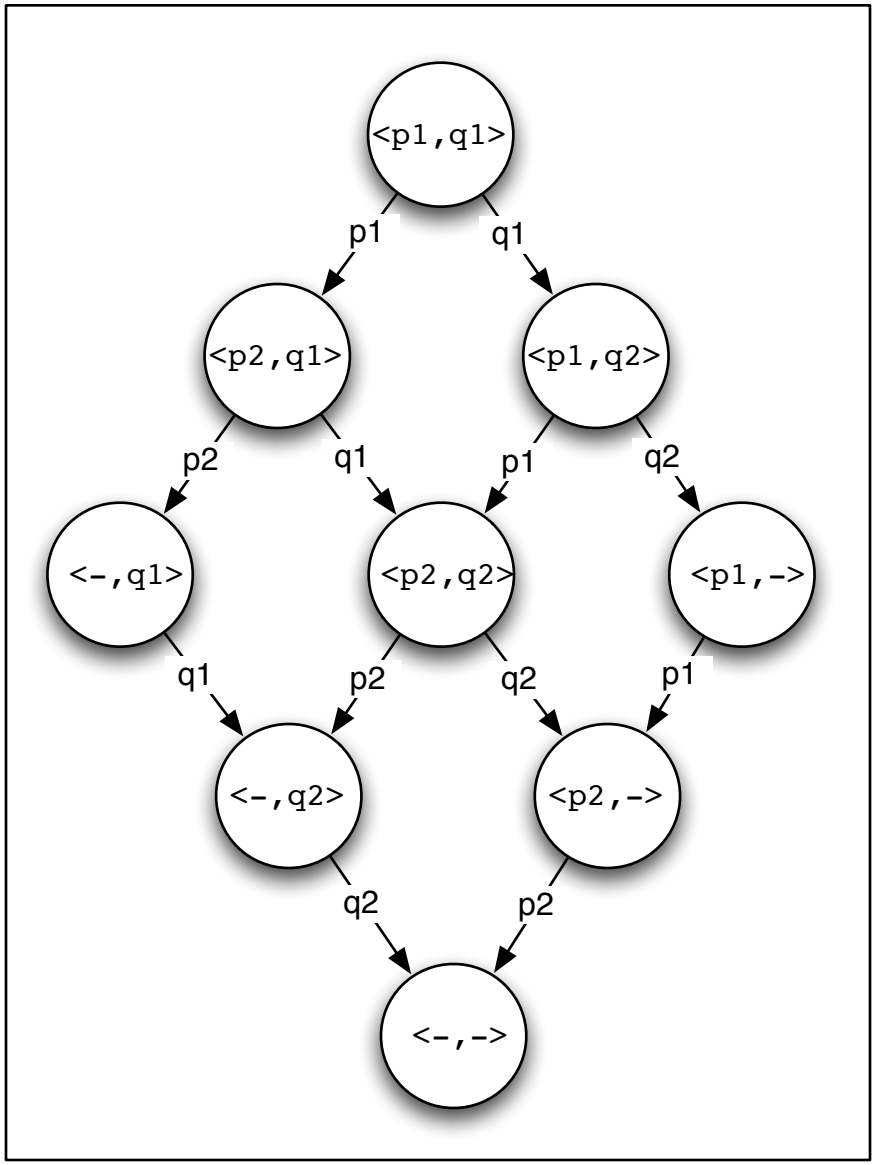
p	q
p1: print(“p1”) **p2:** print(“p2”)	**q1:** print(“q1”) **q2:** print(“q2”)

- State diagram?
- How many scenarios?

EXAMPLE #2

- 6 scenarios:

```
p1 p2 q1 q2
p1 q1 p2 q2
p1 q1 q2 p2
q1 q2 p1 p2
q1 p1 q2 p2
q1 p1 p2 q2
```



EXAMPLE #3

p	q
p1: print(“p1”) **p2:** print(“p2”) **p3:** print(“p3”)	**q1:** print(“q1”) **q2:** print(“q2”) **q3:** print(“q3”)

- Scenarios?

EXAMPLE #3

- 20 scenarios:

```
p1 p2 p3 q1 q2 q3
p1 p2 q1 p3 q2 q3
p1 q1 p2 p3 q2 q3
q1 p1 p1 p3 q2 q3
p1 p2 q1 q2 p3 q3
p1 q1 p2 q2 p3 q3
q1 p1 p2 q2 p3 q3
p1 q1 q2 p2 p3 q3
q1 p1 q2 p2 p3 q3
q1 q2 p1 p2 p3 q3
p1 p2 q1 q2 q3 p3
p1 q1 p2 q2 q3 p3
q1 p1 p2 q2 q3 p3
p1 q1 q2 p2 q3 p3
q1 p1 q2 p2 q3 p3
q1 q2 p1 p2 q3 p3
p1 q1 q2 q3 p2 p3
q1 p1 q2 q3 p2 p3
q1 q2 p1 q3 p2 p3
q1 q2 q3 p1 p2 p3
```

NUM. SCENARIOS WITH 2 PROCESSES

num. actions per proc.	num. scenarios
1	2
2	6
3	20
4	70
5	252
6	924
7	3432
8	12820
...	....

- Note: in these cases process actions have no dependencies...

EXAMPLE #4

- Changing the number of processes: 3 processes..

p	q	r
p1: print(“p1”) **p2:** print(“p2”)	**q1:** print(“q1”) **q2:** print(“q2”)	**r1:** print(“r1”) **r2:** print(“r2”)

- Scenarios?

EXAMPLE #4

- 90 scenarios

```
p1 p2 q1 q2 r1 r2
p1 p2 q1 r1 q2 r2
p1 p2 q1 r1 r2 q2
p1 p2 r1 q1 q2 r2
p1 p2 r1 q1 r2 q2
p1 p2 r1 r2 q1 q2
p1 q1 p2 q2 r1 r2
...
q1 p1 p2 q2 r1 r2
q1 p1 p2 r1 q2 r2
q1 p1 p2 r1 r2 q2
q1 p1 q2 p2 r1 r2
q1 p1 q2 r1 p2 r2
...
r1 p1 p2 q1 q2 r2
r1 p1 p2 q1 r2 q2
r1 p1 p2 r2 q1 q2
r1 p1 q1 p2 q2 r2
r1 p1 q1 p2 r2 q2
...
r1 r2 q1 p1 q2 p2
r1 r2 q1 q2 p1 p2
```

NUM. SCENARIOS WITH 3 PROCESSES

num. actions per proc.	num. scenarios
1	6
2	90
3	1680
4	34650
...	...

GENERALIZING...

- Number of scenarios produced by n processes, each having m_i actions:

$$ns = \frac{(\sum_{i=1}^{n} m_i)!}{\prod_{i=1}^{n} (m_i!)}$$

	$n = 2$	3	4	5	6
$m_i = 2$	6	90	2520	113400	$2^{22.8}$
3	20	1680	$2^{18.4}$	$2^{27.3}$	$2^{36.9}$
4	70	34650	$2^{25.9}$	$2^{38.1}$	$2^{51.5}$
5	252	$2^{19.5}$	$2^{33.4}$	$2^{49.1}$	$2^{66.2}$
6	924	$2^{24.0}$	$2^{41.0}$	$2^{60.2}$	$2^{81.1}$

“THE IMPORTANCE OF BEING ATOMIC”

- Atomic increment (1)

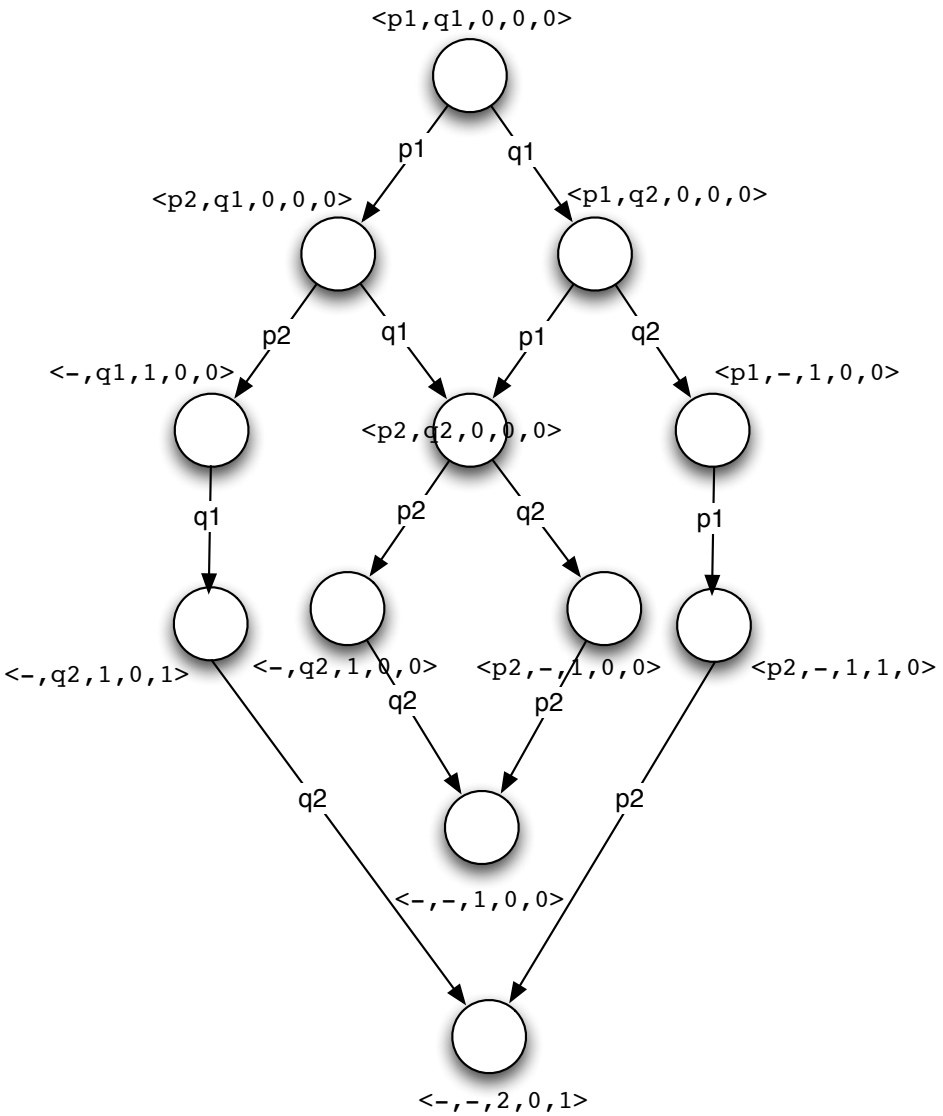
integer n := 0	
p	**q**
p1: n := n + 1	**q1:** n := n + 1

- Non-atomic increment (2)

integer n := 0	
p	**q**
integer tmp; **p1:** tmp := n **p2:** n := tmp + 1	integer tmp; **q1:** tmp := n **q2:** n := tmp + 1

- In the second case, scenarios exist in which the final value of n is 1

NON ATOMIC CASE: STATE DIAGRAM



12 states

Only 2 scenarios over 6 have final n value = 2

ASSIGNMENTS & INCREMENTS AT THE MACHINE-CODE LEVEL

- Stack machines

`integer n := 0`	
p	**q**
p1: `push n` **p2:** `push #1` **p3:** `add` **p4:** `pop n`	**q1:** `push n` **q2:** `push #1` **q3:** `add` **q4:** `pop n`

- Register machines

`integer n := 0`	
p	**q**
p1: `load R1, n` **p2:** `add R1,#n` **p3:** `store n, R1`	**q1:** `load R1, n` **q2:** `add R1,#n` **q3:** `store n, R1`

NON-ATOMIC STRUCTURES (1/3)

- The notion of “atomic” can be referred not only to actions, but also to data structures:
  - a data object is defined *atomic* if it can be in a finite number of states equals to the number of values that it can assume
    - operations change (atomically) that state
  - typically primitive data type in concurrent languages are atomic
    - not always: e.g. `double` in Java
- Abstract data types composed by multiple simpler data objects are typically non atomic
  - es: class in OO languages, structs in C

NON-ATOMIC STRUCTURES (2/3)

- In that case for the ADT (or more generally data object) it is possible to identify two basic types of states: *internal* and *external*
  - the internal state is meaningful for who defines the data object (class)
  - the external state is meaningful for who uses the data object
- The correspondence among internal and external states is *partial*
  - there exist internal states which have no a correspondent external state
  - internal states which have a correspondent external state are defined **consistent**

NON-ATOMIC STRUCTURES (3/3)

- Then, the execution of an operation on a (not-atomic) ADT can go through states that are *not consistent*
  - e.g. a simple list
- This is not a problem in the case of sequential programming
  - thanks to information hiding
- Conversely, it is a problem in the case of concurrent programming
  - it can happen that a process would work on an object while the object is in an inconsistent state, since an process is concurrently operating on it

> it is necessary to introduce proper mechanisms that would guarantee that processes work on data objects that are always in states that are consistent

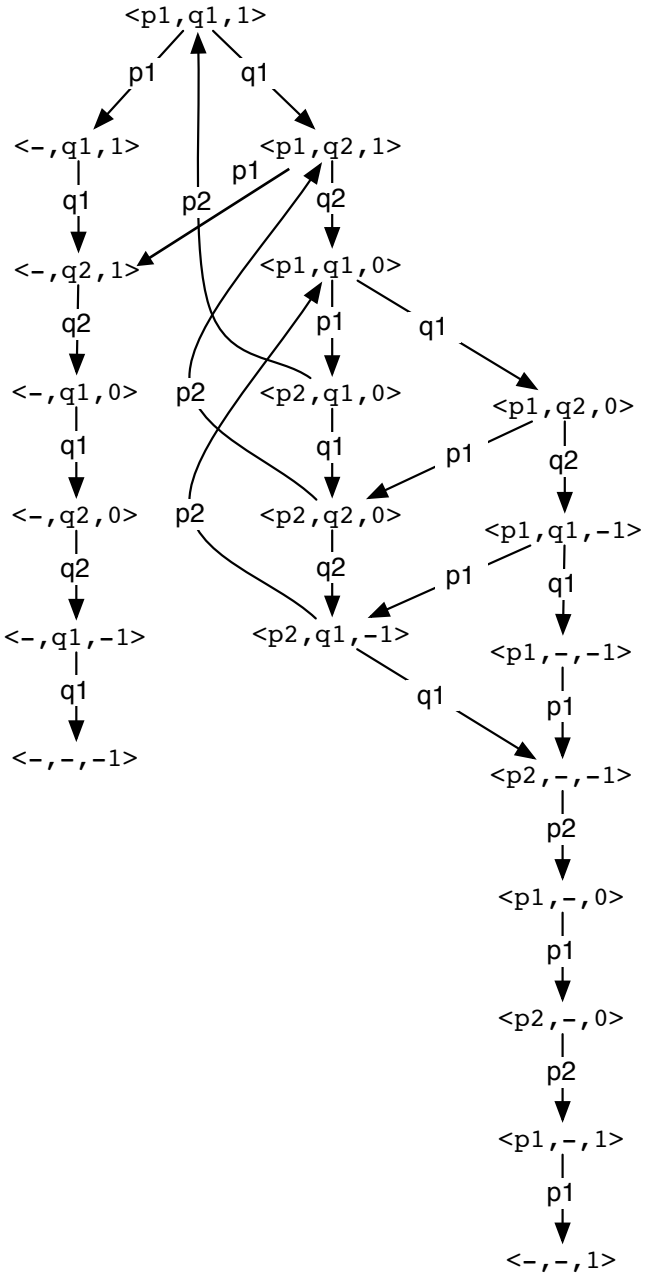
CYCLIC PROCESSES

- **p** and **q** processes cycling on a condition

`integer n := 1`	
p	**q**
p1: `while (n < 1)` **p2:** `n := n + 1`	**q1:** `while (n >= 0)` **q2:** `n := n - 1`

- Analysis
  - state diagram ?
  - construct a scenario in which the loop in p executes exactly one
  - construct a scenario in which the loop in p executes exactly three times

STATE DIAGRAM



IS THIS MODEL A *GOOD* MODEL ?

OR RATHER: IS THE CONCURRENT PROGRAMMING ABSTRACTION JUSTIFIABLE ?

- Actually in the reality computer system **has not a global state**
  - matter of physics
- That's the the role of abstraction: *we create a model of the system in which a kind of global entity executes the concurrent program by arbitrarily interleaving statements*
  - to ease analysis
- But.... is it a valid model for real concurrent computing systems? => *reality check*
  - multitasking systems
  - multicore systems
  - multiprocessor computers
  - distributed systems

ARBITRARILY INTERLEAVING: ABSTRACTING FROM TIME

- **Arbitrary interleaving** means that we ignore time in our analysis of concurrent programs
  - focussing only to
    - partial orders related to action sequences a_1,a_2,...
    - atomicity of the individual action a_j => choosing what is atomic is fundamental
  - robustness with respect to both hardware (processor) and software changes
    - independent from changes in timings / performance
- This makes concurrent programs amenable to *formal analysis*, which is necessary to ensure **correctness** of concurrent programs.
  - proving correctness besides the actual execution time, which is typically strictly dependent on processors speed and system's environment timings

CORRECTNESS OF PROGRAMS

- Checking correctness for sequential programs
  - unit testing based on specified **input** and expecting some specified **output**
    - diagnose, fix, rerun cycle
  - re-running a program with the same input will always give the same result
- Concurrent programming new (challenging) perspective
  - **the same input can give different outputs**, depending on the scenario...
    - some scenarios may give correct output while others do not
  - > we can't debug a concurrent program in the normal way
    - each time you run the program we will likely get a different scenario
- Needs of different kind of approaches
  - formal analysis, *model checking*
  - based on abstract models

CORRECTNESS OF CONCURRENT PROGRAMS

- The correctness of (possibly non-terminating) concurrent programs is defined in terms of **properties** of computations
  - conditions that must be verified in every possible scenarios
- Two type of correctness properties
  - **safety** property
  - **liveness** property

SAFETY PROPERTIES

- The property must be **always** true
  - i.e. for a safety property P to hold, it must be true in *every state* of *every computation*
  - expressed as *invariants* of a computations
- Typically used to specify that “bad things” should never happen
  - mutual exclusion
    - no more than one process is ever present in a critical region
  - no deadlock
    - no process is ever delayed awaiting an event that cannot occur
  - ...

LIVENESS (OR *PROGRESS*) PROPERTIES

- The property must **eventually** *become true*
  - i.e. for a liveness property P to hold, it must be true that *in every* computation there is some state in which P is true
- Typically used to specify that "good things" eventually happen
  - no starvation
    - a process finally gets the resource it needs (CPU time, lock)
  - no dormancy
    - a waiting process is finally awakened
  - reliable communication
    - a message sent by one process to another will be received
  - ...

FAIRNESS

- A liveness property which holds that something good happens infinitely often
- Main example
  - a process activated infinitely often during an application execution, each process getting a fair turn
  - i.e. an action that can be executed, eventually will be executed
    - requirement on the scheduling
- So programs can have different liveness behavior depending on precisely how their instructions are interleaved
  - how instructions are interleaved is a result of a **scheduling policy.**

FAIRNESS & SCHEDULING POLICIES

- **Unconditional Fairness**
  - a scheduling policy is unconditionally fair if every unconditional atomic action that is eligible is executed eventually
- **Weak Fairness**
  - a scheduling policy is weakly fair if it is unconditionally fair and every eligible *conditional* atomic action whose condition becomes and remains true is executed eventually
- **Strong Fairness**
  - a scheduling policy is strongly fair if it is unconditionally fair and every eligible conditional atomic action whose condition becomes true *infinitely often* (infinitely many times) is executed eventually

UNCONDITIONALLY FAIR SCENARIO

- def. **unconditionally fair scenario**
  - a *scenario* is *(unconditionally) fair* if at any state in the scenario a statement that is *continually enabled* eventually appears in the scenario

```
integer n := 0
boolean flag := false
```

p	q
p1: while flag = false **p2:** n := 1 - n	**q1:** flag := true

- Does this algorithm necessarily halt?
  - yes if we assume only fair scenarios
    - if we allow only fair scenario, then eventually an execution of q1 must be included in every scenario
  - **the non-terminating scenario is *not* fair**

CRITICAL SECTION PROBLEM

- Introduced by Dijkstra in 1965, the critical section problem can be considered one of the most important and well-studied problem in concurrent programming, in the context of process competition
- Problem definition
  - N processes, each executing in an infinite loop a sequence of statements that can be divided in 2 subsequences: the *critical section* (CS) and the *non-critical section* (NCS)

```
P[1..n]:
loop {
  <non-critical section>
  entry (or pre-) protocol
  <critical section>
  exit (or post-) protocol
  <non-critical section>
}
```

  - each critical section is typically a sequence of statements that access some shared object.

CRITICAL SECTION PROPERTIES

- Our task is to design *entry and exit protocols* that satisfy the following properties:
  - **mutual exclusion**
    - statements from the critical sections of two or more processes must not be interleaved
  - **freedom from deadlock**
    - if some processes are trying to enter their critical sections, then one of them must eventually succeed
  - **freedom from individual starvation**
    - if any process tries to enter its critical section, then that process must eventually succeed
    - *bounded waiting* property
- Any proposed solution must satisfy also the *progress property* for the CS
  - once a process starts to execute the statements of its critical section, it must eventually finish.
- The NCS need not to progress
  - we can have infinite loops or termination outside the CS

CONCRETE PROBLEMS BASED ON CRITICAL SECTIONS

- The critical section problem is intended to model a system that performs complex computation, but occasionally need to access data or resources (e.g. hardware) that are shared by several process.
- Example
  - a check-in-kiosk at an airport, accessing to a central database of passengers

CRITICAL SECTION SOLUTION: ALGORITHMS VS. MECHANISMS

- The CS problem can be easily solved by using basic mechanisms such as *semaphores* or *locks*, that must be provided by the concurrent machine
  - next modules
- In this module we focus instead on a purely algorithmic solution
  - without using high-level mechanism
  - using solely basic atomic statements (e.g. load and store)

CS FOR TWO PROCESSES

- First we will analyse the problem considering 2 processes P and Q sharing some global variables

Critical section problem	
Global variables	
p	q
Local variables loop forever non-critical section pre-protocol critical section post-protocol	Local variables loop forever non-critical section pre-protocol critical section post-protocol

- The protocols may require local or global variables
  - we assume that no variables used in the CS and NCS are used in the protocols

FIRST ATTEMPT

- Let's consider a first possible solution

First attempt	
Integer turn ← 1	
p	q
loop forever p1: *non-critical section* p2: await turn = 1 p3: *critical section* p4: turn ← 2	loop forever q1: *non-critical section* q2: await turn = 2 q3: *critical section* q4: turn ← 1

- The statement **await** turn = 1 is an implementation independent notation for a statements that waits until the condition turn = 1 becomes true.
  - this can be implemented (albeit inefficiently) by a busy-wait loop that does nothing until the condition is true.

CHECKING CORRECTNESS

- In order to prove the correctness of the solution we construct the state diagram of the concurrent program
  - the algorithm must satisfy the three properties
- To check correctness properties is only necessary to examine the set of reachable states and the transitions among them.
  - states are trip $\langle p_i, q_j, turn \rangle$ he where
    - p_i = current control pointer for process P
    - q_j = current control pointer for process Q
    - $turn$ = content of turn variable
  - the diagram is constructed incrementally starting from the initial state and considering what the potential states are
    - 16 states
- Describing the mutual exclusion property
  - the mutual exclusion correctness property holds if the set of all accessible states does not contain a state of the form `<p3,q3,_>`.

REDUCING THE NUMBER OF

- For studying correctness, we can reduce the number of states without altering the semantics of the program
  - by removing unessential statements
- Whatever statements are executed by the CS and in the NCS are totally irrelevant to the correctness of the synchronisation algorithms, so they can be removed

First attempt (abbreviated)	
Integer turn ← 1	
p	q
loop forever p1: await turn = 1 p2: turn ← 2	loop forever q1: await turn = 2 q2: turn ← 1

- The mutual exclusion properties holds if there are not states of of the kind <p2,q2,_>
  - building the new state diagram => only 4 states

CHECKING CORRECTNESS (2)

- To check the correctness we have to check that the three properties hold
  - mutual exclusion is satisfied
    - there are not scenarios including states of the kind <p2,q2,_>
  - freedom of deadlock is satisfied
    - if some processes are trying to enter the CS (executing the await), then one eventually succeeds
  - freedom of starvation is **not satisfied**
    - a process that wants to enter in CS (pre-protocol) can starve while waiting a process which engaged an infinite loop inside its NCS (where there is not the progress property)
- Turn function as permission resource to enter the CS
  - if the process holding the permission remains indefinitely in its non-critical section, the other process will never receive the resource and will never enter its critical section
    - both processes sets and tests a single global variable: if one process dies, the other is blocked.
- Therefore the solution is *not correct*

SECOND ATTEMPT

- We introduce separate want variables intended to indicate when a process is in its CS

Second attempt

boolean wantp ← false
boolean wantq ← false

p	q
loop forever p1: *non-critical section* p2: await wantq = false p3: wantp ← true p4: *critical section* p5: wantp ← false	loop forever q1: *non-critical section* q2: await wantp = false q3: wantq ← true q4: *critical section* q5: wantq ← false

- Mutual exclusion: not having scenarios with p4 and q4

SECOND ATTEMPT ABBREVIATED

- Removing irrelevant statements..

Second attempt: Abbreviated

boolean wantp ← false
boolean wantq ← false

p	q
loop forever p1: await wantq = false p2: wantp ← true p3: wantp ← false	loop forever q1: await wantp = false q2: wantq ← true q3: wantq ← false

- Mutual exclusion: not having scenarios with p3 and q3

CORRECTNESS OF THE 2nd ATTEMPT

- By constructing the state diagrams it is simple to see that the mutual exclusion property is not satisfied
  - the state <p3,p3,true,true> is reachable.
- The problem is due to the non-atomicity of the pre-protocol and post-protocol

THIRD ATTEMPT...

- Previous problem can be solved by considering the await statement part of the CS and moving the assignment before the await

Third attempt	
boolean wantp ← false boolean wantq ← false	
p	q
loop forever p1: *non-critical section* p2: wantp ← true p3: await !wantq p4: *critical section* p5: wantp ← false	loop forever q1: *non-critical section* q2: wantq ← true q3: await !wantp q4: *critical section* q5: wantq ← false

- Correctness of the 3rd attempt
  - the mutual exclusion problem is solved, however the solution suffers of possible deadlocks
    - both processes are trying to enter in their CS but neither will ever do
  - more precisely, this is a *livelock*

FOURTH ATTEMPT...

- Deadlock occurs when both processes simultaneously insist on entering their CS
  - we can solve the problem by requiring a process to give up its intention to enter its CS if it discovers that it is contending with the other process

Fourth attempt

```
boolean wantp ← false
boolean wantq ← false
```

p	q
loop forever p1: *non-critical section* p2: wantp ← true p3: while wantq p4: wantp ← false p5: wantp ← true p6: *critical section* p7: wantp ← false	loop forever q1: *non-critical section* q2: wantq ← true q3: while wantp q4: wantq ← false q5: wantq ← true q6: *critical section* q7: wantq ← false

- Correctness of the 4th attempt
  - the livelock is solved, but there is still the possibility of starvation, in the case of perfect interleaving in executing while...

DEKKER'S ALGORITHM

- The problem can be solved with a simple variation of the 4th algorithm, requiring that *the right to insist* on entering - instead of the right to enter - is explicitly passed between the processes by means of the variable turn
  - this solution was found for the first time by the dutch mathematician T.J. Dekker, in 1965
  - Dekker's algorithm for solving the CS problem can be then considered a combination of the first and fourth attempt

A SOLUTION: DEKKER'S ALGORITHM

Dekker's algorithm

```
boolean wantp ← false
boolean wantq ← false
integer turn ← 1
```

p	q
loop forever p1: *non-critical section* p2: wantp ← true p3: while wantq p4: if turn = 2 p5: wantp ← false p6: await turn = 1 p7: wantp ← true p8: *critical section* p9: turn ← 2 p10: wantp ← false	loop forever q1: *non-critical section* q2: wantq ← true q3: while wantp q4: if turn = 1 q5: wantq ← false q6: await turn = 2 q7: wantq ← true q8: *critical section* q9: turn ← 1 q10: wantq ← false

- Dekker's algorithm is correct
  - it satisfies mutual exclusion, and it is free from deadlock and starvation.

PETERSON'S ALGORITHM

- Other solutions have been proposed through time, improving Dekker's one:
  - Peterson (1981), Manna-Pnueli, Doran-Thomas
- Peterson's solution is more concise, by collapsing the two await statements into one with a compound condition

```
boolean wantp := false;
boolean wantq := false;
integer turn := 1;
```

p	q
loop forever p1: NCS p2: wantp := true p3: turn := 2 p4: await !wantq || turn = 1 p5: CS p6: wantp := false	loop forever q1: NCS q2: wantq := true q3: turn := 1 q4: await !wantp || turn = 2 q5: CS q6: wantq := false

INTRODUCING COMPOUND ATOMIC STATEMENTS

- Dekker's algorithm works on any architecture, providing just load and store as atomic statements.
- Actually, the algorithm and – more generally – the CS problem and mutual exclusion problems can be greatly simplified if we can exploit more complex atomic statements
  - directly provided by the concurrent machine
- Main examples are the *test-and-set*, *exchange*, *fetch-and-add*, *compare-and-swap*
  - test-and-set is an atomic statement defined as the execution of the two following statements with no possibility of interleaving

```
test-and-set (x,r):
< r := x, x := 1 >
```

  - a commonly used notation for specifying atomicity is to put angled bracket around the group of statements

CRITICAL SECTION WITH TEST-AND-

```
integer lock := 0;
```

p	q

integer was_locked
 	
loop forever
p1: NCS
    repeat
p2:   test_and_set(lock,was_locked)
p3: until was_locked = 0
p4: CS
p5: lock := 0
	integer was_locked
	
	loop forever
	q1: NCS
	    repeat
	q2: test_and_set(lock,was_locked)
	q3: until was_locked = 0
	q4: CS
	q5: lock := 0

LOCKS

- By exploiting compound atomic statement we can easily realize a basic *lock* mechanism
  - simple solution for the CS problem
  - protecting critical section of code
- Two stages / atomic operations
  - *acquiring* the lock
  - *releasing* the lock

```
lock sharedlock;
```

p	q
loop forever p1: NCS p2: acquire(sharedlock) p3: CS p4: release(sharedlock)	loop forever q1: NCS q2: acquire(sharedlock) q3: CS q4: release(sharedlock)

ATOMIC STATEMENT IN JAVA: `synchronized`

- Atomic statements in Java can be implemented by means `synchronized` blocks using the same shared `lock` object

```
// process (thread) A
...
synchronized (lock) {
 <statement a>
 <statement b>
 ..
}
...
```

```
// process (thread) B
...
synchronized (lock) {
 <statement c>
 <statement d>
 ..
}
...
```

  - in this way the sequence of actions a.b and c.d are executed without interleaving the individual actions
    - there cannot exist scenarios with sequence like a.c.b.d or c.a.d.b

CS SOLUTION WITH TICKETS: BAKERY ALGORITHM (TIE-

- An alternative solution to CS problem for N-process accounts for introducing a *ticket* to establish the turn of a process

N-process solution for CS with tickets

`int num ← 1`
`int next ← 1`
`turn[1:n] ← [0,0,0,...]`

p[i]

`loop forever`
`p1: NCS`
`p2: < turn[i] ← num; num ← num + 1 >`
`p3: await turn[i] = next`
`p4: CS`
`p5: next ← next + 1`

- Potential shortcomings: arithmetic overflow for `turn` and `num`
  - monotonically growing

ABOUT CORRECTNESS: VERIFICATION VS. TESTING

- **Testing**
  - activity looking for *failures*
    - revealing faults
- **Verification**
  - checking if the system satisfies its specification
    - "have we built the system *right*?"
- **Validation**
  - checking if the system satisfy customer's expectation
    - "have we built the *right system*?"

Fault

- a *manifestation* of an error
  - error = (human) action the produces an incorrect result

Failure

- when a fault is encountered, it may result in a failure
  - a failure may be caused by many faults
  - a fault can cause many failures

LIVESS AND SAFETY TESTS

- Two kinds of tests
  - **Tests of safety**
    - verify that a class or system behavior conforms to its specification
    - usually taking the form of testing **invariants**
    - typically using *assertions*
  - **Tests of liveness**
    - progress / non-progress tests
    - very hard to set up
- Problems & limits of testing
  - typically checking only a subset of the scenarios
  - *“heisenbugs”* phenomenon
    - test code can introduce timing or synchronisation artefacts that can mask bugs

TESTING VS. VERIFICATION

- Testing is aimed at verifying that a (correctness) property holds for some selected scenario
  - *testing reveals the presence of errors, not their absence*
- Verification is aimed at verifying that a property holds **for all possible scenarios (traces)**
  - need of *formal* techniques

FORMAL METHODS

- Two principal (classes of) formal techniques:
  - **model checking**
    - where verification is done by generating one by one all the states of the systems and by checking the properties state by state
    - can be automated by model checkers tools
  - **inductive proofs of invariants**
    - invariant properties are proved by *induction* over the states of the system
    - can be automated by tools called deductive systems
- Both techniques rely on some kind of *formal language / calculus to specify correctness properties*

CORRECTNESS PROPERTIES IN PROPOSITION CALCULUS

- With propositional calculus, correctness properties are expressed as *logic formulae* that must be true in order to verify the property in some state of the system
  - formulae are assertions obtained by composing propositions through logic connectors
    - and, or, not, implications, equivalence
- In our case propositions are about *the values of the variables* and *of the control pointers* during an execution of a concurrent programming
  - e.g. given the boolean variable `wantp`, an atomic proposition (assertion) `wantp` is true in a certain state if and only if the value of the variable `wantp` is true in that state
- Each label of a statement of a process will be used as an atomic proposition whose interpretation is "the control pointer of that process is currently at that label"
  - e.g. p1 proposition asserts that the control pointer of the process p is at the label p1

AN EXAMPLE: MUTUAL EXCLUSION

Third attempt	
boolean wantp ← false boolean wantq ← false	
p	q
loop forever p1: *non-critical section* p2: wantp ← true p3: await !wantq p4: *critical section* p5: wantp ← false	loop forever q1: *non-critical section* q2: wantq ← true q3: await !wantp q4: *critical section* q5: wantq ← false

- Formula
  - is tru$p_4 \wedge q_4$h control pointers of the processes are in the critical section
- if it exists some state in which this formula is true, then it means that *the mutual exclusion property is **not** satisfied*

> dually, a program *satisfies the mutual exclusion property* if the formula $\neg(p_4 \wedge q_4)$ is true for *every possible state of every scenario*

TEMPORAL LOGIC

- Processes and systems change their state over the time, and then also the interpretation of formulae about their state can change over the time.
  - > we need a formal language/calculus that would take this aspect into the account
  - > *temporal logic* is one of the most basic and popular one
    - *Amir Pnueli, "The Temporal Logic of Programs", 18th Annual Symposium on Foundations of Computer Science, 1977*
- The **temporal logic** is a formal logic obtained by adding temporal operators to propositional or predicate logic
  - **Linear Temporal Logic (LTL)**
    - to express properties that must be true (at a state) for *every possible scenario*
    - linear / discrete model of time
  - **Branching Temporal Logics**
    - to express properties that must be true in some or all scenarios starting from a state
    - an example: **CTL (computational tree logic)**

LTL: TEMPORAL OPERATORS

- LTL is based on two basic temporal operators: *always* and *eventually*
  - **box** or **always** temporal operator: □A
    - the formula $\Box A$ is true in a state s_i of a computation if and only if the formula A is true in *all* states s_j with $j >= i$
      - synonym: □p = **G p** (Globally p)
    - the always operator can be used then to specify *safety properties*, because it specifies what must be always be true
  - **diamond** or **eventually** temporal operator: ◇A
    - the formula $\Diamond A$ is true in a state s_i of a computation if and only if the formula A is true in *some* states s_j with $j >= i$
      - synonym: ◇p = **F p** (Finally p)
    - the eventually operator is used to specify *liveness properties*, because it specifies something that eventually be true

BASIC PROPERTIES

- Reflexivity: $\Box A \rightarrow A$

  $A \rightarrow \Diamond A$

- Duality: $\neg \Box A = \Diamond \neg A$

  $\neg \Diamond A = \Box \neg A$

- Sequences of operators: $\Diamond \Box A$

  $\Box \Diamond A$

DEDUCTION WITH TEMPORAL LOGICS

- Temporal logic is a formal system of deductive logic with its own axioms and rules of inference
  - it can be used to formalise the semantics of concurrent programs and used to rigorously prove correctness properties of programs
- An example of a theorems in TL:

$(\Diamond\Box A1 \wedge \Diamond\Box A2) \rightarrow \Diamond\Box (A1 \wedge A2)$ is true.

$(\Box\Diamond A1 \wedge \Box\Diamond A2) \rightarrow \Box\Diamond (A1 \wedge A2)$ is false

SPECIFYING SAFETY PROPERTIES

- Box operator can be used to specify safety properties
  - as properties that must be always true
- $\Box P$, where $P = \neg Q$ and Q is the description of a bad state
  - an example: mutual exclusion in CS problem

First attempt	
Integer turn ← 1	
p	q
loop forever p1: *non-critical section* p2: await turn = 1 p3: *critical section* p4: turn ← 2	loop forever q1: *non-critical section* q2: await turn = 2 q3: *critical section* q4: turn ← 1

  - mutual exclusion property: $\Box\neg(p_3 \wedge q_3)$

SPECIFYING LIVENESS PROPERTIES

- Diamond operator can be used to specify liveness properties
  - as conditions that eventually will be true
- $\Diamond P$, where P is the description of a good case
  - an example: *progress property (no starvation)* in CS problem

First attempt	
Integer turn ← 1	
p	q
loop forever p1: *non-critical section* p2: await turn = 1 p3: *critical section* p4: turn ← 2	loop forever q1: *non-critical section* q2: await turn = 2 q3: *critical section* q4: turn ← 1

  - progress property for *current state*: $p_2 \rightarrow \Diamond p_3$
  - progress property *for all states*: $\Box(p_2 \rightarrow \Diamond p_3)$

NEXT OPERATOR

- Besides always and eventually, another unary operator is defined in LTL, called next: ○φ
  - ○φ is true if property φ holds in next state, in every possible scenario
    - also represented by X, i.e. X φ (from neXt)

BINARY OPERATORS

- Always and eventually are unary operators. An example of useful and frequently used binary operator is until
  - **Until** operator: A **U** B
    - A U B is true in a state Si if and only if B is true in some state Sj, j>=i and A is true in all state Sk, i<=k<j.
    - That is: eventually B becomes true and that A is true until that happens
  - **Weak-Until** operator: A **W** B
    - like Until operator, but formula B is not required to become true eventually. If it does not, A must remain true indefinitely
    - A W B = as long as A is false, B must be true

REMARKS

- Always and eventually operators can be defined in term of until:
  - $\Diamond A = true\ U\ A$ (eventually A becomes true)
  - $\Box A = \neg \Diamond (\neg A)$ (A always remains true)

OVERTAKING

- Consider the following scenario in the CS problem

```
try-p,try-q,CSq,try-q,CSq,...,CSq,CSp
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                1000 times
```

- It's not an example of starvation...
  - it is true that ◇CSp
  - > but it's evident too that freedom from starvation can be a very weak property!
- in some cases we want to ensure that a process would enter its critical section within a reasonable amount of time

K-BOUNDED OVERTAKING PROPERTY

- ***k*-bounded-overtaking property**
  - from the time a process p attempts to enter its critical section, another process can enter at most *k* times before p does
  - Example: 3-overtaking
    - `try-p,try-q,CSq,try-q,CSq,try-q,CSq,CSp`

- The property can be expressed by the weak until operator W
  – example: 1-bounded overtaking

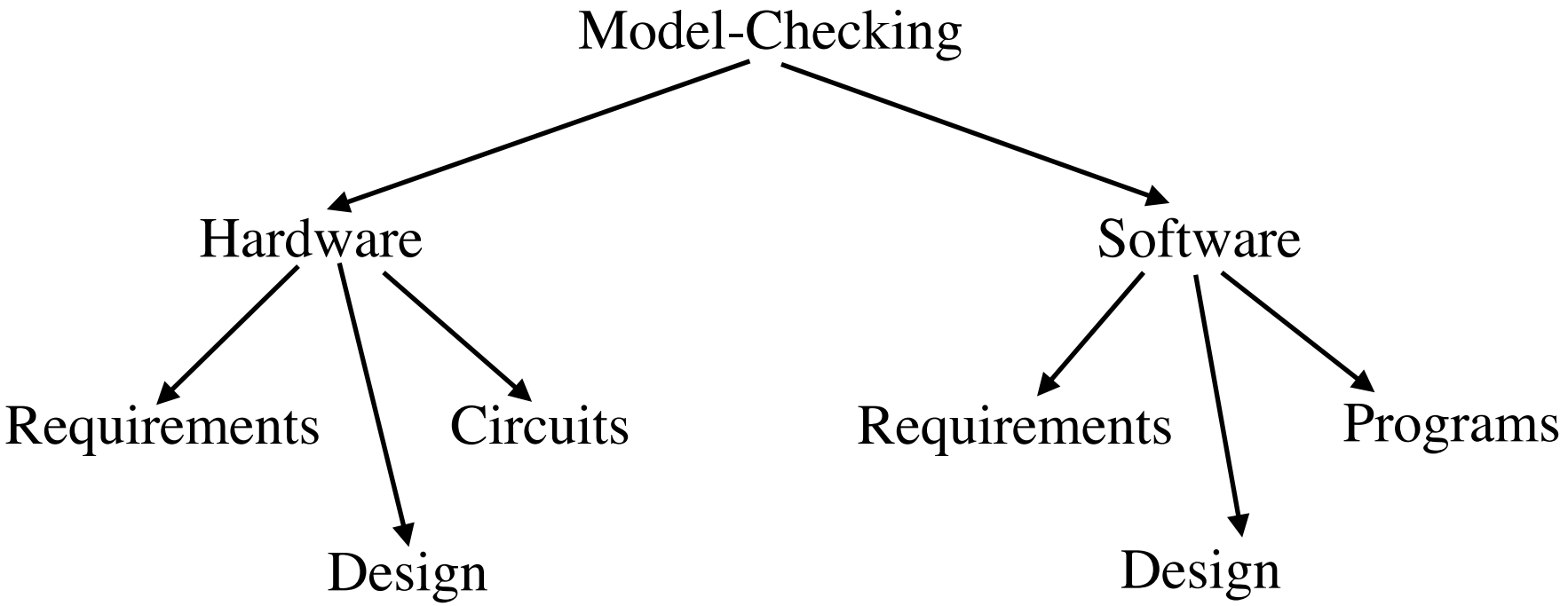
`try_cs(P) => not cs(Q) W cs(Q) W not cs(Q) W cs(P)`
(with pars: `try_cs(P) => not cs(Q) W (cs(Q) W (not cs(Q) W cs(P)))`

any position satisfying try_cs(P) (that is P is in try_cs(P)) is followed by an interval in which Q is not in cs, followed by an interval in which Q is in cs, followed by an interval in which Q is again not in cs, which can be terminated only by a state in which P is in cs. It therefore implies that from the time P starts waiting at try_cs, there can be at most one continuous interval in which Q is in cs, before P is admitted to cs.

VERIFICATION TECHNIQUES (1/2):

- **Model checking** is the most important and used technique for automatically checking correctness properties of concurrent systems
  - invaluable conceptual and practical tool for software engineers
- Strategy based on *exhaustively searching the entire state space* of a system and verify if certain properties are satisfied
  - properties as predicates on a system state or states, expressed as a logical specification such as propositional temporal logic formula
  - if the system satisfies the property, the model checker generates a *confirmation* response
    - otherwise, it produces a *trace* (counterexample) => useful also to identify bugs, not only to prove correctness
- SW vs. HW model checking
  - can be applied also to hardware
  - e.g. Intel adopting Model-Checking after the Pentium Bug in 1994
  - used in mission critical software systems
    - e.g. NASA after Mars Polar Lander incident in 1999

MODEL-CHECKING APPLICATIONS



- Program model checking
  - application of the model-checking techniques to software systems
    - in particular to the final implementation
    - discovering software defects

DEALING WITH THE STATE-SPACE EXPLOSION PROBLEM

- The big problem of model-checking technique is the **size of the state space**
  - how to manage graph of millions of states? Is it feasible ?
- State-of-the art techniques
  - applying rules to reduce the number of states
    - using variables that can be modelled by a limited number of values
  - incremental construction of the whole graph
    - exploring only reachable state of an execution
    - checking the truth of a correctness specification as the incremental diagram is constructed, stopping the construction is a falsifying state is found
  - *symbolic* model checking
    - working with set of states

SPIN AND PROMELA

- **SPIN** is a widely used model-checker used in both academic research and industrial software development
  - extremely efficient
  - used in modelling and designing concurrent and distributed systems in general
- **PROMELA** is the language that is used in Spin to write concurrent programs modelling language
  - limited number of constructs intended to be used to build models of concurrent systems

AN EXAMPLE: DEKKER IN PROMELA

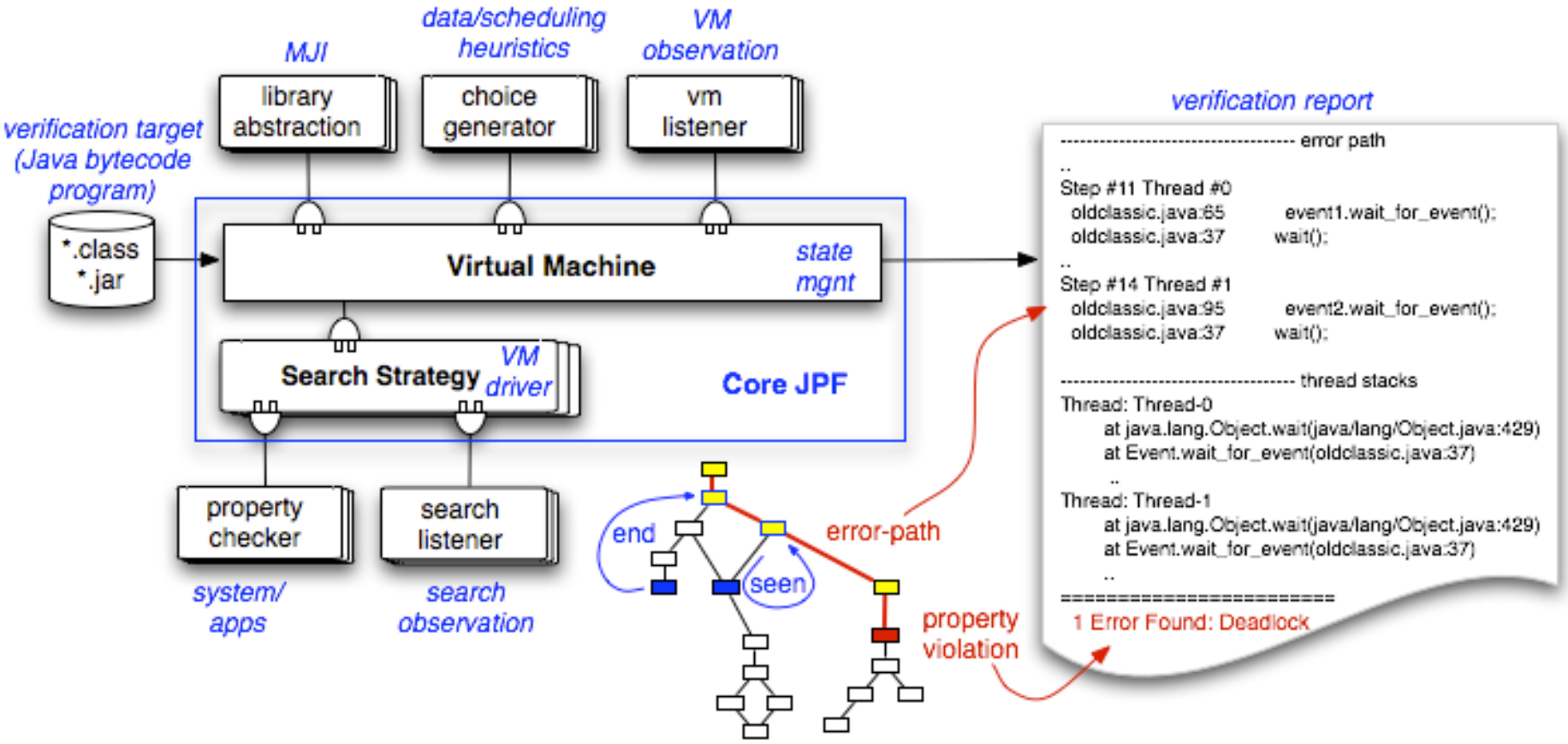
```
bool    wantp = false, wantq = false;
byte    turn = 1;

proctype p() {
    do ::
        wantp = true;
        do :: !wantq -> break;
           :: else ->
            if :: (turn == 1)
               :: (turn == 2) ->
                   wantp = false;
                   (turn == 1);
                   wantp = true
            fi
        od;
        printf("LOG: p in CS\n");
        turn = 2;
        wantp = false
    od
}

proctype q() { /* similar */}

init {
  atomic {
    run  p
    run  q
  }
}
```

JAVA PATH FINDER (JPF)

- JPF is a recent model-checker specialized for the verification of programs written in Java
  - developed by NASA, used for critical software
  - open-source project
    - `http://javapathfinder.sourceforge.net/`

- JPF is a special JVM executing programs theoretically along all possible scenarios (execution paths), checking for property violations
  - deadlocks, uncaught exceptions, etc
  - If it finds an error, JPF reports the whole execution that leads to it

JPF MODEL OF OPERATION



VERIFICATION TECHNIQUES (2/2): INDUCTIVE PROOF OF INVARIANTS

- **invariant**
  - a formula that must be invariably true at any point of any computation
    - e.g. $\neg(p_4 \wedge q_4)$
- Invariants can be proved using **induction** over the states of all the computations:
  - to prove that A is an invariant:
    - prove that A is true in the initial state (*the base case*)
    - assume that A is true in a generic state S *(inductive hypothesis)* and prove that A is true in all the possible state next to S *(inductive step)*
- *Deductive systems*
  - software systems for automated theorem proving

SAFETY AND LIVENESS PROPERTY VERIFICATION

- Safety property are easier to verify
  - a safety property must be true at all states
    - it is sufficient to find a state not verifying the property to complete the verification
  - a liveness property claims that a state satisfying a property will inevitably occur
    - it is not sufficient to check states one by one, it is necessary to check all possible scenarios
    - > it requires more complex theory and software techniques

TEMPORAL LOGIC OF ACTIONS (TLA/TLA+)

- A formal specification language introduced by Leslie Lamport
  - based on simple discrete math (set theory and predicates)
  - http://research.microsoft.com/en-us/um/people/lamport/tla/tla.html
- Used to specify and verify complex real-world systems
  - es: Amazon Cloud
    - "Use of Formal Methods at Amazon Web Services”
      - http://research.microsoft.com/en-us/um/people/lamport/tla/amazon.html

TLA+ SPEC

- A TLA+ specification describes the set of all possible legal behaviors (execution traces) of a system.
  - can be used to describe both
    - the desired correctness properties of the system (the ‘what’)
    - the design of the system (the ‘how’)
- TLA+ objective: make it possible to show that a system design correctly implements the desired correctness properties
  - either via conventional mathematical reasoning
  - by using tools such as the TLC model checker
    - a tool which takes a TLA+ specification and exhaustively checks the desired correctness properties across all of the possible execution traces
- PlusCal (formerly called +CAL)
  - an algorithm language based on TLA+
  - for writing algorithms, just as a programming language is for writing programs
    - a PlusCal algorithm is translated to a TLA+ specification, which can be checked with the TLC model checker

PLUSCAL EXAMPLE

```
--algorithm Peterson {
   variables flag = [i ∈ {0,1} ↦ FALSE], turn = 0;
   process (proc ∈ {0,1}) {
        a0:  while (TRUE) {
        a1:      flag[self] := TRUE;
        a2:      turn := Not(self);
       a3a:      if (flag[Not(self)]) {goto a3b} else {goto cs} ;
       a3b:      if (turn = Not(self)) {goto a3a} else {goto cs} ;
        cs:      skip; \* critical section
        a4:      flag[self] := FALSE;
             } \* end while
   } \* end process
 } \* end algorithm
```

Fig. 1. Peterson's algorithm in PlusCal.

$$MutualExclusion \triangleq (pc[0] \neq \text{“cs”}) \vee (pc[1] \neq \text{“cs”})$$

$$\text{THEOREM } Spec \Rightarrow \Box MutualExclusion$$

BIBLIOGRAPHY

- “Principles of Concurrent and Distributed Programming” - M. Ben-Ari. Addison-Wesley
  - chapter 1, 2, 3, 4
- Further suggested reading & references
  - “What good temporal logic is?” - Lamport, 1983
  - “Transition Systems” - M. Nygaard, 2004
  - “Principle of Model Checking” - Baier & Katoen